

UNIVERSIDADE DO VALE DO ITAJAÍ, CAMPUS ITAJAÍ
ENGENHARIA DE COMPUTAÇÃO

LARISSA DE SOUSA GOUVEA

THIAGO YUKIO HORITA PACHECO

Trabalho M2 STR

ITAJAÍ
2024

Questões:

1) Você(s) deverá(ão) desenvolver o software requisitado pela empresa, seguindo a especificação sobre sensores, controladores e threads. Você deve implementar o sistema no padrão microkernel com múltiplas tasks. Você deve aplicar um dos conceitos de distribuição de prioridades das tasks: DM, RM ou Importância. Justifique sua escolha.

Conceitos das distribuições de prioridades das tasks:

Rate Monotonic (RM - Taxa Monotônica): Atribui maior prioridade às tarefas com períodos menores. Essa abordagem é ideal quando as tarefas são periódicas e o deadline é igual ao período, sendo uma das políticas de escalonamento mais eficientes para garantir deadlines sob condições de carga baixa.

Deadline Monotonic (DM - Deadline Monotônico): Prioriza tarefas com deadlines relativos menores. DM é semelhante ao RM quando o deadline é igual ao período e é considerado ótimo em situações onde um conjunto de tarefas pode ser escalonado usando essa política. Ele organiza as prioridades de acordo com o prazo final mais próximo, mantendo uma prioridade fixa.

Importância: Neste modelo, a tarefa com maior importância na aplicação recebe a prioridade mais alta. A determinação da importância depende do contexto específico da aplicação, mas geralmente prioriza tarefas que têm um impacto crítico sobre o funcionamento ou a segurança do sistema.

A escolha da distribuição de prioridades foi baseada na importância, permitindo atender de forma eficaz às demandas do sistema, especialmente aquelas que impactam a segurança e o desempenho do veículo. O monitoramento do airbag é atribuído à prioridade mais alta, pois, em caso de colisão, precisa ser acionado imediatamente para proteger os ocupantes. O monitoramento do ABS (prioridade 9) é crucial para evitar o travamento das rodas durante a frenagem, garantindo a estabilidade do veículo. A injeção eletrônica (prioridade 7) é vital para otimizar o desempenho do motor com ajustes rápidos na mistura de ar e combustível.

O monitoramento da temperatura (prioridade 4) é importante para prevenir superaquecimento do motor. O monitoramento do cinto de segurança (prioridade 3) é relevante para a segurança dos ocupantes, enquanto o monitoramento da velocidade (prioridade 2) e do consumo (prioridade 2) fornecem informações úteis, mas não críticas. A atualização do display (prioridade 1) recebe a menor prioridade, já que sua função não impacta diretamente a segurança ou o desempenho imediato do veículo. Essa estrutura de prioridades permite que o sistema responda rapidamente em situações de emergência.

3) Você deve especificar o modelo de tarefa identificando (modelo básico visto em aula):

- a. Tempo de execução
- b. Tempo de Interferência. Pode ser adquirido utilizando a distribuição de prioridades.
- c. Período de ocorrência (pensando em periódico, esporádico ou aperiódico) no quesito tempo
- d. Deadline

4) Você deve analisar o Worst Case Execution Time e Worst Case Response Time das tarefas.

- e. WCET
- f. WCRT

Monitoramento de Injeção Eletrônica:

- a. Valor médio foi de 62us
- b. Valor Médio foi de 174us
- c. Esporádico
- d. 500us
- e. 78us
- f. 264us

Monitoramento de Temperatura do Motor:

- a. Valor médio foi de 72us
- b. Valor Médio foi de 264us
- c. Esporádico
- d. 20ms
- e. 98us
- f. 338ms

Monitoramento de ABS:

- a. Valor Médio foi de 78us
- b. Valor Médio foi de 62us
- c. Esporádico
- d. 100ms
- e. 111us
- f. 174us

Monitoramento de Airbag:

- a. Valor Médio foi de 48us
- b. Por ela ser a task com maior prioridade, em nosso código, ela não irá ter Tempo de Interferência
- c. Esporádico
- d. 100ms

- e. 50us
- f. 62us

Monitoramento de Cinto de Segurança:

- a. Valor Médio foi de 315us
- b. Valor Médio foi de 383us
- c. Aperiódico
- d. 1s
- e. 588us
- f. 971us

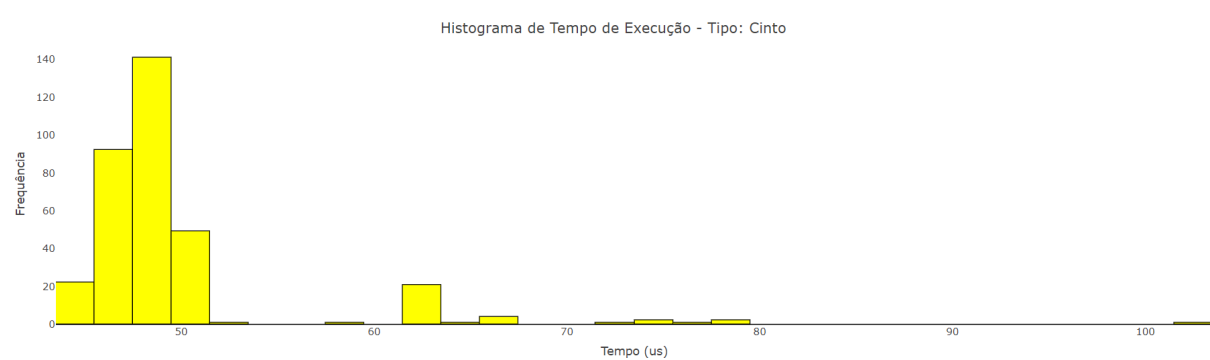
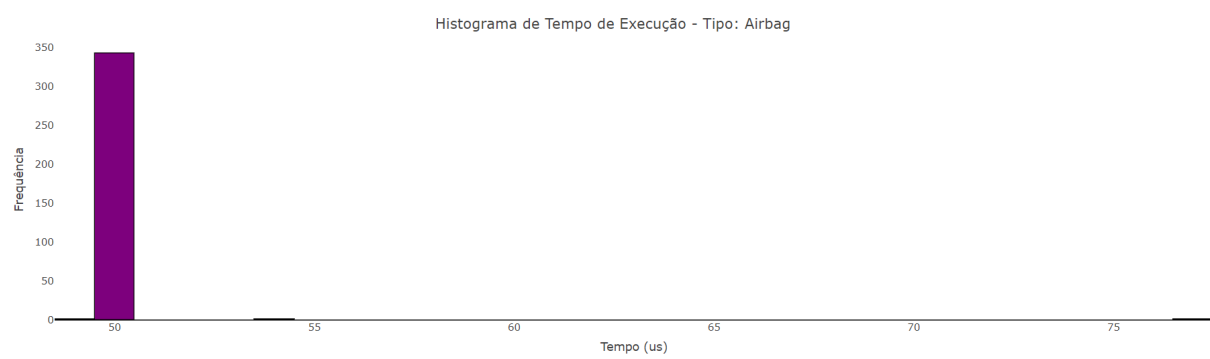
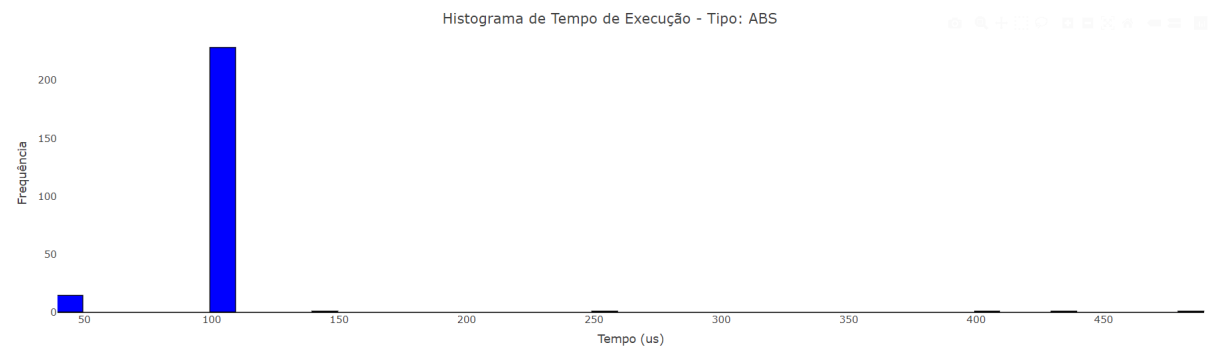
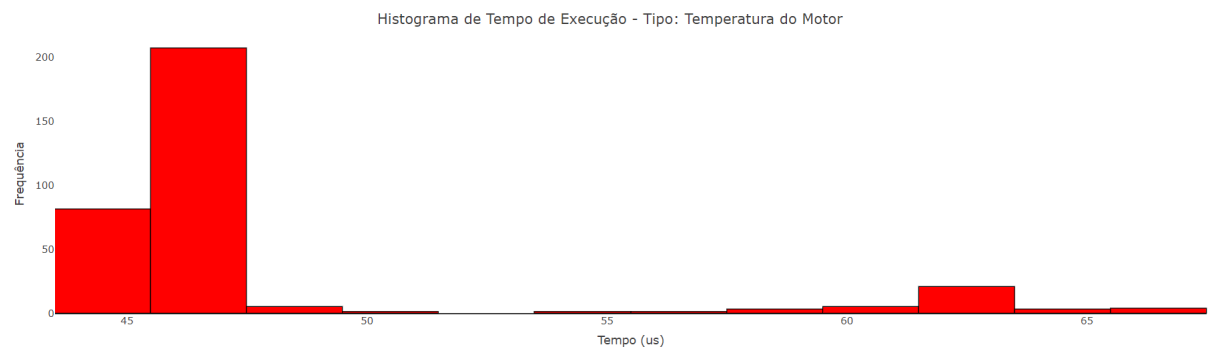
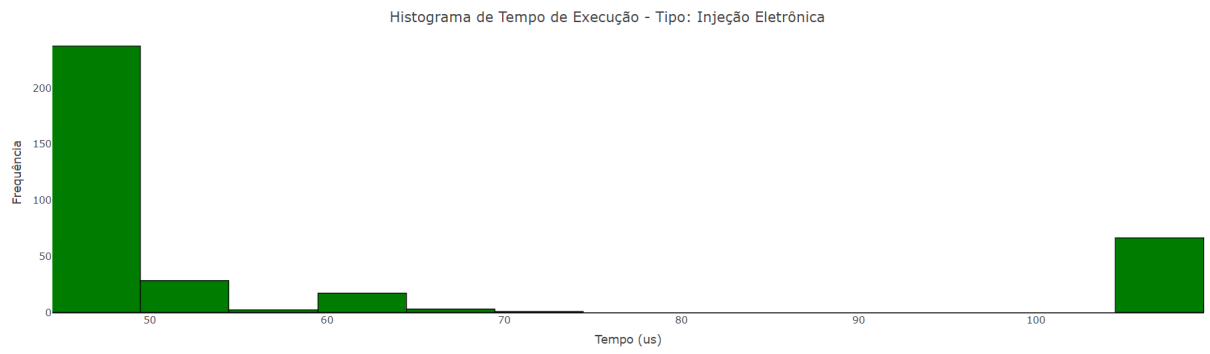
WCET da Injeção Eletrônica: 78 us
HWM da Injeção Eletrônica: 90 us
WCRT da Injeção Eletrônica: 264 us

WCET da Temperatura do Motor: 98 us
HWM da Temperatura do Motor: 119 us
WCRT da Temperatura do Motor: 383 us

WCET do ABS: 111 us
HWM do ABS: 112 us
WCRT do ABS: 174 us

WCET do Airbag: 50 us
HWM do Airbag: 62 us
WCRT do Airbag: 62 us

WCET do Cinto: 584 us
HWM do Cinto: 588 us
WCRT do Cinto: 971 us



```

library(shiny)
library(plotly)
library(dplyr)

setwd("C:/SistemasEmTempoReal/Visualizacoes")
dados <- read.table(
  "Teste_tabela.csv", header = TRUE, sep = ",", dec = "."
)

ui <- fluidPage(
  br(),
  lapply(unique(dados$tipo), function(t) {
    tags$div(
      plotlyOutput(outputId = paste0("histograma_", t)),
      style = "margin-bottom: 30px;"
    )
  })
)

server <- function(input, output, session) {

  categorias <- unique(dados$tipo)
  cores_categoria <- setNames(c(rep(NA, length(categorias))), categorias)

  cores_categoria["Cinto"] <- "yellow"
  cores_categoria["Injeção Eletrônica"] <- "green"
  cores_categoria["Airbag"] <- "purple"
  cores_categoria["Temperatura do Motor"] <- "red"
  cores_categoria["ABS"] <- "blue"

  lapply(categorias, function(t) {
    output[[paste0("histograma_", t)]] <- renderPlotly({
      plot_ly(
        data = dados[dados$tipo == t, ],
        x = ~tempo.us.,
        type = 'histogram',
        name = t,
        marker = list(
          color = cores_categoria[t],
          line = list(
            color = 'black',
            width = 1
          )
        )
      )
    }) %>%
    layout(
      title = paste("Histograma de Tempo de Execução - Tipo:", t),
      yaxis = list(
        title = "Frequência",
        showgrid = FALSE
      ),
    ),
  })
}

```

```
      xaxis = list(
        title = "Tempo (us)"
      ),
      showlegend = FALSE
    )
  })
}

shinyApp(ui, server)
```

5) Indicar as características do hardware que podem afetar o tempo de execução e tempo de resposta.

As características do hardware que podem afetar o tempo de execução e o tempo de resposta são as interferências e as possíveis outras tarefas que estão acontecendo ao mesmo tempo, já que as tasks com baixa prioridade podem sofrer mais com interrupções, a velocidade do processador, tendo em vista que a frequência de operação afeta diretamente a ESP32, a memória cache, pois a ESP32 possui uma memória cache pequena, assim caso as instruções e os dados das tarefas não estiverem na memória cache o tempo de execução será maior, e o tempo de comunicação do sistema com os sensores.

6) Utilize uma abordagem de medições identificando o HWM(100%) e também aplique o conceito de (m,k)-firme com 10 ativações de janela para cada tarefa. Usem como quantidade de medições de tempo pelo menos 20 amostras por tarefa. Também indique o fator skip entre perda de deadlines

Task	Deadline	m	k	Fator Skip
Injeção	500us	7	10	Máximo de três falha
Temperatura	20ms	6	10	Máximo de quatro falhas
ABS	100ms	8	10	Máximo de duas falhas
Airbag	100ms	9	10	Máximo de uma falha
Cinto	1s	5	10	Máximo de cinco falhas

No programa foi feito um contador em cada task, este contador só adiciona valor a si mesmo quando entrar na condição que o sensor desta task foi detectado, assim quando ocorrer 20 ativações será feito a checagem do modelo de (m, k)-firme, onde é comparado o valor de tempos de execuções que passaram o deadline com k menos m, multiplicado pelo index, este índice serve para que o programa continue funcionando conforme mais ativações ocorre, caso o contador de tempos de execuções que ultrapassam o deadline forem maior que k menos m, então é feito o print da mensagem que houve uma falha no modelo, caso o valor seja menor, então é feito o print avisando que a tarefa está dentro dos limites definidos do modelo.

8) Devem ser identificadas as tasks com requisitos temporais hard e soft, e descrever o que levou a essa análise. Além disso, as operações podem ser consideradas sem requisito temporal (essa escolha deve ser embasada). Lembre-se de atualizar as prioridades de execução das threads do sistema no qual você faz uso.

Task de Monitoramento da Injeção Eletrônica

Requisito temporal hard, Em situações críticas, como frenagens ou manobras rápidas, a resposta rápida do sistema de injeção é essencial para evitar falhas que poderiam comprometer a segurança do veículo.

Task de Monitoramento da Temperatura do Motor

O monitoramento da temperatura do motor é considerado um requisito temporal soft. Embora a temperatura precise ser monitorada continuamente para evitar superaquecimento e danos ao motor, os impactos de uma falha nesse monitoramento não são imediatamente críticos.

Task de Controle de ABS

Requisito temporal hard, o ABS é um sistema de segurança fundamental que precisa ser acionado rapidamente em situações de frenagem de emergência. O deadline de 100 ms é necessário para garantir que o sistema atue a tempo.

Task de Controle do Airbag

Requisito temporal hard, o acionamento do airbag é crítico para a segurança e precisa responder imediatamente a uma colisão. Qualquer atraso pode comprometer a segurança dos ocupantes, por isso o deadline de 100 ms é considerado hard.

Task de Monitoramento do Cinto de Segurança

Requisito temporal soft, a segurança não é comprometida se houver um pequeno atraso, então ela é classificada como requisito temporal soft.

Task de Atualização do Display

Requisito temporal soft, a atualização do display é necessária para fornecer feedback ao usuário, mas pequenos atrasos não causam falha crítica no sistema. Ela tem um deadline de 1 segundo, então é classificada como soft.

Tasks de Monitoramento de Velocidade e Consumo Médio

Sem requisito temporal, o cálculo de velocidade e consumo médio não interfere em decisões críticas em tempo real e pode ser processado sem um deadline rígido.

```
void app_main(void)
{
    xTaskCreate(monitoramento_injecao, "monitoramento_injecao", 2048,
NULL, 7, NULL); //Prioridade mais alta
    xTaskCreate(monitoramento_temperatura,
"monitoramento_temperatura", 2048, NULL, 4, NULL); //Prioridade média
    xTaskCreate(monitoramento_abs, "monitoramento_abs", 2048, NULL,
9, NULL); //Prioridade alta
    xTaskCreate(monitoramento_airbag, "monitoramento_airbag", 2048,
NULL, 12, NULL); //Prioridade alta
    xTaskCreate(monitoramento_cinto, "monitoramento_cinto", 2048,
NULL, 3, NULL); //Prioridade baixa
    xTaskCreate(atualizar_display, "atualizar_display", 2048, NULL,
1, NULL); // Prioridade mais baixa
    xTaskCreate(monitoramento_velocidade, "monitoramento_velocidade",
2048, NULL, 2, NULL); // Prioridade baixa
    xTaskCreate(monitoramento_consumo, "monitoramento_consumo", 2048,
NULL, 2, NULL); // Prioridade baixa
}
```

Código Funcional:

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/queue.h"
#include "esp_log.h"
#include "driver/gpio.h"
#include "esp_timer.h"
#include "driver/touch_pad.h"
#include "soc/rtc_periph.h"
#include "soc/sens_periph.h"

#include <math.h>

static const char *TAG = "Touch pad";

#define TOUCH_THRESH_NO_USE    (0)
#define TOUCH_THRESH_PERCENT   (80)
#define TOUCHPAD_FILTER_TOUCH_PERIOD (10)

#define DEADLINE_INJECAO 500 // 500 us
#define DEADLINE_TEMPERATURA 20000 // 20 ms
#define DEADLINE_ABS 100000 // 100 ms
#define DEADLINE_AIRBAG 100000 // 100 ms
#define DEADLINE_CINTO 1000000 // 1 segundo

#define GPIO_ATUADOR_INJECAO GPIO_NUM_18
#define GPIO_ATUADOR_TEMPERATURA GPIO_NUM_19
#define GPIO_ATUADOR_ABS GPIO_NUM_21
#define GPIO_ATUADOR_AIRBAG GPIO_NUM_22
#define GPIO_ATUADOR_CINTO GPIO_NUM_23

#define AMOSTRAS 200 // Número de amostras para cálculo da média

static bool motor_ativo = false;
static bool frenagem_ativo = false;
static bool vida_ativa = false;

static float amostras_velocidade[AMOSTRAS];
static float amostras_consumo[AMOSTRAS];
static int contagem_velocidade = 0;
static int contagem_consumo = 0;
```

```
static float velocidade_media = 0;
static float consumo_media = 0;

// static bool s_pad_activated[TOUCH_PAD_MAX];
static uint32_t s_pad_init_val[TOUCH_PAD_MAX];

volatile bool TOUCH_INJECAO = false;
volatile bool TOUCH_TEMPERATURA = false;
volatile bool TOUCH_ABS = false;
volatile bool TOUCH_AIRBAG = false;
volatile bool TOUCH_CINTO = false;

volatile uint64_t HWM_INJECAO = 0;
volatile uint64_t HWM_TEMPERATURA = 0;
volatile uint64_t HWM_ABS = 0;
volatile uint64_t HWM_AIRBAG = 0;
volatile uint64_t HWM_CINTO = 0;

volatile uint64_t WCET_INJECAO = 0;
volatile uint64_t WCET_TEMPERATURA = 0;
volatile uint64_t WCET_ABS = 0;
volatile uint64_t WCET_AIRBAG = 0;
volatile uint64_t WCET_CINTO = 0;

volatile uint64_t WCRT_INJECAO = 0;
volatile uint64_t WCRT_TEMPERATURA = 0;
volatile uint64_t WCRT_ABS = 0;
volatile uint64_t WCRT_AIRBAG = 0;
volatile uint64_t WCRT_CINTO = 0;

volatile uint64_t TEMPO_INTERFERENCIA_INJECAO = 0;
volatile uint64_t TEMPO_INTERFERENCIA_TEMPERATURA = 0;
volatile uint64_t TEMPO_INTERFERENCIA_ABS = 0;
volatile uint64_t TEMPO_INTERFERENCIA_AIRBAG = 0;
volatile uint64_t TEMPO_INTERFERENCIA_CINTO = 0;

void config_atuadores() {
    gpio_set_direction(GPIO_ATUADOR_INJECAO, GPIO_MODE_OUTPUT);
    gpio_set_direction(GPIO_ATUADOR_TEMPERATURA, GPIO_MODE_OUTPUT);
    gpio_set_direction(GPIO_ATUADOR_ABS, GPIO_MODE_OUTPUT);
    gpio_set_direction(GPIO_ATUADOR_AIRBAG, GPIO_MODE_OUTPUT);
    gpio_set_direction(GPIO_ATUADOR_CINTO, GPIO_MODE_OUTPUT);
}
```

```

}

static void tp_example_set_thresholds(void)
{
    uint16_t touch_value;
    for (int i = 0; i < TOUCH_PAD_MAX; i++) {
        //read filtered value
        touch_pad_read_filtered(i, &touch_value);
        s_pad_init_val[i] = touch_value;
        ESP_LOGI(TAG, "test init: touch pad [%d] val is %d", i,
touch_value);
        //set interrupt threshold.
        ESP_ERROR_CHECK(touch_pad_set_thresh(i, touch_value * 2 /
3));
    }
}

static void monitoramento_injecao(void *pvParameter) {
    touch_pad_intr_enable();

    uint64_t start_time = 0, execution_time = 0, total_time = 0/*,
tempo_interferencia = 0, tempo_resposta = 0*/;
    // uint64_t total_execution_time = 0, avg_execution_time = 0;
    const int m = 7, k = 10;
    int count_m = 0, index = 2, count = 0;

    while (1) {
        if (TOUCH_INJECAO) {
            start_time = esp_timer_get_time(); // Marca o início da
execução

            vTaskDelay(pdMS_TO_TICKS(1) / 1000);
            vTaskDelay(pdMS_TO_TICKS(10) / 1000);
            gpio_set_level(GPIO_ATUADOR_INJECAO, 1);
            vTaskDelay(pdMS_TO_TICKS(5) / 1000);
            gpio_set_level(GPIO_ATUADOR_INJECAO, 0);
            vTaskDelay(pdMS_TO_TICKS(10) / 1000);

            execution_time = esp_timer_get_time() - start_time;

            TEMPO_INTERFERENCIA_INJECAO = HWM_AIRBAG + HWM_ABS;

```

```

        total_time = esp_timer_get_time() - start_time;
        if(total_time > HWM_INJECAO) {
            HWM_INJECAO = total_time;
        }

        // Atualiza o tempo total de execução
        // total_execution_time += execution_time;

        // Atualiza o WCET se o tempo atual de execução for o
maior registrado
        if (execution_time > WCET_INJECAO) {
            WCET_INJECAO = execution_time;
        }

        printf("\033[32mTempo de execução da Injeção Eletrônica:
%llu us\033[0m\n", execution_time);

        count++;

        if(execution_time > DEADLINE_INJECAO) {
            count_m++;
        }

        if ((HWM_INJECAO + TEMPO_INTERFERENCIA_INJECAO) >
WCRT_INJECAO) {
            WCRT_INJECAO = HWM_INJECAO +
TEMPO_INTERFERENCIA_INJECAO;
        }

        if((count % 20) == 0) {
            if((count_m) > (k - m)*index) {
                printf("\033[32mFalha de firme (m, k): mais
deadlines perdidos que o permitido.\n");
            }
            else{
                printf("\033[32mTarefa dentro dos limites de (m,
k)-firme.\n");
            }
            index++;
        }
        TOUCH_INJECAO = false;

```

```

    }
    vTaskDelay(pdMS_TO_TICKS(100));
}
}

static void monitoramento_temperatura(void *pvParameter) {
    touch_pad_intr_enable();

    uint64_t start_time = 0, execution_time = 0, total_time = 0/*,
tempo_interferencia = 0, tempo_resposta = 0*/;
    // uint64_t total_execution_time = 0, avg_execution_time = 0;
    const int m = 6, k = 10;
    int count_m = 0, index = 2, count = 0;

    while (1) {
        if (TOUCH_TEMPERATURA) {
            start_time = esp_timer_get_time();

            vTaskDelay(pdMS_TO_TICKS(1) / 1000);
            vTaskDelay(pdMS_TO_TICKS(10) / 1000);
            gpio_set_level(GPIO_ATUADOR_TEMPERATURA, 1);
            vTaskDelay(pdMS_TO_TICKS(5) / 1000);
            gpio_set_level(GPIO_ATUADOR_TEMPERATURA, 0);
            vTaskDelay(pdMS_TO_TICKS(10) / 1000);

            execution_time = esp_timer_get_time() - start_time;

            // Atualiza o tempo total de execução
            // total_execution_time += execution_time;

            // Atualiza o WCET se o tempo atual de execução for o
maior registrado
            if (execution_time > WCET_TEMPERATURA) {
                WCET_TEMPERATURA = execution_time;
            }

            TEMPO_INTERFERENCIA_TEMPERATURA = HWM_AIRBAG + HWM_ABS +
HWM_INJECAO;

            total_time = esp_timer_get_time() - start_time;
            if (total_time > HWM_TEMPERATURA) {
                HWM_TEMPERATURA = total_time;
            }
        }
    }
}

```

```

    }

    printf("\033[31mTempo de execução da Temperatura do
Motor: %llu us\033[0m\n", execution_time);

    count++;

    if(execution_time > DEADLINE_TEMPERATURA) {
        count_m++;
    }

    if ((HWM_TEMPERATURA + TEMPO_INTERFERENCIA_TEMPERATURA) >
WCRT_TEMPERATURA) {
        WCRT_TEMPERATURA = (HWM_TEMPERATURA +
TEMPO_INTERFERENCIA_TEMPERATURA);
    }

    if((count % 20) == 0) {
        if((count_m) > (k - m)*index) {
            printf("\033[31mFalha de firme (m, k): mais
deadlines perdidos que o permitido.\n");
        }
        else{
            printf("\033[31mTarefa dentro dos limites de (m,
k)-firme.\n");
        }
        index++;
    }
    TOUCH_TEMPERATURA = false;
}
vTaskDelay(pdMS_TO_TICKS(100));
}
}

static void monitoramento_abs(void *pvParameter) {
    touch_pad_intr_enable();

    uint64_t start_time = 0, execution_time = 0, total_time = 0/*,
tempo_interferencia = 0, tempo_resposta = 0*/;
    // uint64_t total_execution_time = 0, avg_execution_time = 0;
    const int m = 8, k = 10;
    int count_m = 0, index = 2, count = 0;

```

```

while (1) {
    if (TOUCH_ABS) {
        start_time = esp_timer_get_time();

        vTaskDelay(pdMS_TO_TICKS(1) / 1000);
        vTaskDelay(pdMS_TO_TICKS(10) / 1000);
        gpio_set_level(GPIO_ATUADOR_ABS, 1);
        vTaskDelay(pdMS_TO_TICKS(5) / 1000);
        gpio_set_level(GPIO_ATUADOR_ABS, 0);
        vTaskDelay(pdMS_TO_TICKS(10) / 1000);

        execution_time = esp_timer_get_time() - start_time;

        // Atualiza o tempo total de execução
        // total_execution_time += execution_time;

        // Atualiza o WCET se o tempo atual de execução for o
maior registrado
        if (execution_time > WCET_ABS) {
            WCET_ABS = execution_time;
        }

        TEMPO_INTERFERENCIA_ABS = HWM_AIRBAG;

        total_time = esp_timer_get_time() - start_time;
        if(total_time > HWM_ABS) {
            HWM_ABS = total_time;
        }

        printf("\033[34mTempo de execução do ABS: %llu
us\033[0m\n", execution_time);

        count++;

        if(execution_time > DEADLINE_ABS) {
            count_m++;
        }

        if ((HWM_ABS + TEMPO_INTERFERENCIA_ABS) > WCRT_ABS) {
            WCRT_ABS = (HWM_ABS + TEMPO_INTERFERENCIA_ABS);
        }
    }
}

```

```

        if((count % 20) == 0) {
            if((count_m) > (k - m)*index) {
                printf("\033[34mFalha de firme (m, k): mais
deadlines perdidos que o permitido.\n");
            }
            else{
                printf("\033[34mTarefa dentro dos limites de (m,
k)-firme.\n");
            }
            index++;
        }
        TOUCH_ABS = false;
    }
    vTaskDelay(pdMS_TO_TICKS(100));
}

static void monitoramento_airbag(void *pvParameter) {
    touch_pad_intr_enable();

    uint64_t start_time = 0, execution_time = 0, total_time = 0/*,
tempo_interferencia = 0, tempo_resposta = 0*/;
    // uint64_t total_execution_time = 0, avg_execution_time = 0;
    const int m = 9, k = 10;
    int count_m = 0, index = 2, count = 0;

    while (1) {
        if (TOUCH_AIRBAG) {
            start_time = esp_timer_get_time();

            vTaskDelay(pdMS_TO_TICKS(1) / 1000);
            vTaskDelay(pdMS_TO_TICKS(10) / 1000);
            gpio_set_level(GPIO_ATUADOR_AIRBAG, 1);
            vTaskDelay(pdMS_TO_TICKS(5) / 1000);
            gpio_set_level(GPIO_ATUADOR_AIRBAG, 0);
            vTaskDelay(pdMS_TO_TICKS(10) / 1000);

            execution_time = esp_timer_get_time() - start_time;

            // Atualiza o tempo total de execução
            // total_execution_time += execution_time;

```

```

        // Atualiza o WCET se o tempo atual de execução for o
maior registrado
        if (execution_time > WCET_AIRBAG) {
            WCET_AIRBAG = execution_time;
        }

        TEMPO_INTERFERENCIA_AIRBAG = 0;

        total_time = esp_timer_get_time() - start_time;
        if (total_time > HWM_AIRBAG) {
            HWM_AIRBAG = total_time;
        }

        printf("\033[35mTempo de execução do Airbag: %llu
us\033[0m\n", execution_time);

        count++;

        if (execution_time > DEADLINE_AIRBAG) {
            count_m++;
        }

        if ((HWM_AIRBAG + TEMPO_INTERFERENCIA_AIRBAG) >
WCRT_AIRBAG) {
            WCRT_AIRBAG = (HWM_AIRBAG +
TEMPO_INTERFERENCIA_AIRBAG);
        }

        if ((count % 20) == 0) {
            if ((count_m) > (k - m) * index) {
                printf("\033[35mFalha de firme (m, k): mais
deadlines perdidos que o permitido.\n");
            }
            else {
                printf("\033[35mTarefa dentro dos limites de (m,
k)-firme.\n");
            }
            index++;
        }
        TOUCH_AIRBAG = false;
    }

```

```

        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

static void monitoramento_cinto(void *pvParameter) {
    touch_pad_intr_enable();

    uint64_t start_time = 0, execution_time = 0, total_time = 0/*,
tempo_interferencia = 0, tempo_resposta = 0*/;
    // uint64_t total_execution_time = 0, avg_execution_time = 0;
    const int m = 5, k = 10;
    int count_m = 0, index = 2, count = 0;

    while (1) {

        if (TOUCH_CINTO) {
            start_time = esp_timer_get_time(); // Marca o início da
execução

            vTaskDelay(pdMS_TO_TICKS(1) / 1000);
            vTaskDelay(pdMS_TO_TICKS(10) / 1000);
            gpio_set_level(GPIO_ATUADOR_CINTO, 1);
            vTaskDelay(pdMS_TO_TICKS(5) / 1000);
            gpio_set_level(GPIO_ATUADOR_CINTO, 0);
            vTaskDelay(pdMS_TO_TICKS(10) / 1000);

            // Calcula o tempo de execução
            execution_time = esp_timer_get_time() - start_time;

            // Atualiza o WCET se o tempo atual de execução for o
maior registrado
            if (execution_time > WCET_CINTO) {
                WCET_CINTO = execution_time;
            }

            total_time = esp_timer_get_time() - start_time;
            if (total_time > HWM_CINTO) {
                HWM_CINTO = total_time;
            }

            TEMPO_INTERFERENCIA_CINTO = HWM_AIRBAG + HWM_ABS +
HWM_INJECAO + HWM_TEMPERATURA;

```

```

        printf("\033[33mTempo de execução do Cinto: %llu
us\033[0m\n", execution_time);

        count++;

        if(execution_time > DEADLINE_CINTO) {
            count_m++;
        }

        if ((HWM_CINTO + TEMPO_INTERFERENCIA_CINTO) > WCRT_CINTO)
{
            WCRT_CINTO = (HWM_CINTO + TEMPO_INTERFERENCIA_CINTO);
        }

        if((count % 20) == 0) {
            if((count_m) > (k - m)*index) {
                printf("\033[33mFalha de firme (m, k): mais
deadlines perdidos que o permitido.\n");
            }
            else{
                printf("\033[33mTarefa dentro dos limites de (m,
k)-firme.\n");
            }
            index++;
        }
        TOUCH_CINTO = false;
    }
    vTaskDelay(pdMS_TO_TICKS(100));
}

void monitoramento_velocidade(void *pvParameter) {
    while (1) {
        if (contagem_velocidade < AMOSTRAS) {
            float velocidade = (float)(rand() % 100);
            amostras_velocidade[contagem_velocidade++] = velocidade;
        } else {
            float soma = 0;
            for (int i = 0; i < AMOSTRAS; i++) {
                soma += amostras_velocidade[i];
            }
        }
    }
}

```

```

        velocidade_media = soma / AMOSTRAS;
        contagem_velocidade = 0;
    }
    vTaskDelay(pdMS_TO_TICKS(100));
}

void monitoramento_consumo(void *pvParameter) {
    while (1) {
        if (contagem_consumo < AMOSTRAS) {
            float consumo = (float)(rand() % 15);
            amostras_consumo[contagem_consumo++] = consumo;
        } else {
            float soma = 0;
            for (int i = 0; i < AMOSTRAS; i++) {
                soma += amostras_consumo[i];
            }
            consumo_media = soma / AMOSTRAS;
            contagem_consumo = 0;
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

void atualizar_display(void *pvParameter) {
    while (1) {
        printf("\n\nVelocidade média: %.2f km/h\n",
velocidade_media);
        printf("Consumo médio: %.2f L/100km\n\n", consumo_media);

        printf("\033[32mWCET da Injeção Eletrônica: %llu
us\033[0m\n", WCET_INJECAO);
        printf("\033[32mHWM da Injeção Eletrônica: %llu us\033[0m\n",
HWM_INJECAO);
        printf("\033[32mWCRT da Injeção Eletrônica: %llu
us\033[0m\n\n", WCRT_INJECAO);

        printf("\033[31mWCET da Temperatura do Motor: %llu
us\033[0m\n", WCET_TEMPERATURA);
        printf("\033[31mHWM da Temperatura do Motor: %llu
us\033[0m\n", HWM_TEMPERATURA);
        printf("\033[31mWCRT da Temperatura do Motor: %llu

```

```

us\033[0m\n\n", WCRT_TEMPERATURA);

    printf("\033[34mWCET do ABS: %llu us\033[0m\n", WCET_ABS);
    printf("\033[34mHWM do ABS: %llu us\033[0m\n", HWM_ABS);
    printf("\033[34mWCRT do ABS: %llu us\033[0m\n\n", WCRT_ABS);

    printf("\033[35mWCET do Airbag: %llu us\033[0m\n",
WCET_AIRBAG);
    printf("\033[35mHWM do Airbag: %llu us\033[0m\n",
HWM_AIRBAG);
    printf("\033[35mWCRT do Airbag: %llu us\033[0m\n\n",
WCRT_AIRBAG);

    printf("\033[33mWCET do Cinto: %llu us\033[0m\n",
WCET_CINTO);
    printf("\033[33mHWM do Cinto: %llu us\033[0m\n", HWM_CINTO);
    printf("\033[33mWCRT do Cinto: %llu us\033[0m\n\n",
WCRT_CINTO);

    // Reseta o estado dos subsistemas para o próximo ciclo
    motor_ativo = false;
    frenagem_ativo = false;
    vida_ativa = false;

    vTaskDelay(pdMS_TO_TICKS(1000)); // Atualiza a cada 1
segundo
}
}

static void tp_example_rtc_intr(void *arg)
// void IRAM_ATTR touch_pad_isr_handler(void *arg)
{
    uint32_t pad_intr = touch_pad_get_status();
    //clear interrupt
    touch_pad_clear_status();

    if((pad_intr >> 0) & 0x01) {
        TOUCH_INJECAO = true;
    }
    if((pad_intr >> 3) & 0x01) {
        TOUCH_TEMPERATURA = true;
    }
}

```

```

    if((pad_intr >> 4) & 0x01) {
        TOUCH_ABS = true;
    }
    if((pad_intr >> 7) & 0x01) {
        TOUCH_AIRBAG = true;
    }
    if((pad_intr >> 9) & 0x01) {
        TOUCH_CINTO = true;
    }
}

static void tp_example_touch_pad_init(void)
{
    for (int i = 0; i < TOUCH_PAD_MAX; i++) {
        //init RTC IO and mode for touch pad.
        touch_pad_config(i, TOUCH_THRESH_NO_USE);
    }
}

void app_main(void)
{
    config_atuadores();
    // Initialize touch pad peripheral, it will start a timer to run
a filter
    ESP_LOGI(TAG, "Initializing touch pad");
    touch_pad_init();
    // If use interrupt trigger mode, should set touch sensor FSM
mode at 'TOUCH_FSM_MODE_TIMER'.
    touch_pad_set_fsm_mode(TOUCH_FSM_MODE_TIMER);
    // Set reference voltage for charging/discharging
    // For most usage scenarios, we recommend using the following
combination:
    // the high reference voltage will be 2.7V - 1V = 1.7V, The low
reference voltage will be 0.5V.
    touch_pad_set_voltage(TOUCH_HVOLT_2V7, TOUCH_LVOLT_0V5,
TOUCH_HVOLT_ATTEN_1V);
    // Init touch pad IO
    tp_example_touch_pad_init();
    // Initialize and start a software filter to detect slight change
of capacitance.
    touch_pad_filter_start(TOUCHPAD_FILTER_TOUCH_PERIOD);
    // Set thresh hold

```

```
tp_example_set_thresholds();
// Register touch interrupt ISR
touch_pad_isr_register(tp_example_rtc_intr, NULL);

// Modo de interrupção, ativa a interrupção de toque
xTaskCreate(monitoramento_injecao, "monitoramento_injecao", 2048,
NULL, 7, NULL);
xTaskCreate(monitoramento_temperatura,
"monitoramento_temperatura", 2048, NULL, 4, NULL);
xTaskCreate(monitoramento_abs, "monitoramento_abs", 2048, NULL,
9, NULL);
xTaskCreate(monitoramento_airbag, "monitoramento_airbag", 2048,
NULL, 12, NULL);
xTaskCreate(monitoramento_cinto, "monitoramento_cinto", 2048,
NULL, 3, NULL);
xTaskCreate(atualizar_display, "atualizar_display", 2048, NULL,
1, NULL); // Prioridade mais baixa
xTaskCreate(monitoramento_velocidade, "monitoramento_velocidade",
2048, NULL, 2, NULL); // Prioridade baixa
xTaskCreate(monitoramento_consumo, "monitoramento_consumo", 2048,
NULL, 2, NULL); // Prioridade baixa
}
```